

Lab 1: Requirements Engineering and UML Use-Case Modelling

Problem Statement 04 - Student Project Portfolio and Showcase App

Name Bharat V Bhat
SRN PES1UG24CS113
Course / Section Software Engineering, Section B
Faculty Dr. Sapna V M

1. Problem Overview

Annual project expos require a centralized platform where student teams submit multimedia project artifacts, panel judges grade the projects using digital rubrics, and members of the public vote for projects under strict anti-sybil (anti-fraud) protection. This document records the requirements, the use-case model, and one detailed use-case flow for the proposed system.

Actors

- Student Team - creates the showcase profile, uploads project artifacts, and views results.
- Panel Judge - grades assigned projects using a digital rubric.
- Public Voter - casts a verified public vote and views the leaderboard.

2. Software Requirements

The table below lists five functional requirements (FR) and two non-functional requirements (NFR). FR-001 and NFR-001 are the requirements given in the problem statement; the remaining requirements were derived from the scenario.

Req ID	Type	Description	Priority	Acceptance Criteria	Rationale
FR-001	Functional	The system shall allow student teams to create showcase profiles containing an abstract summary, a demo video link, and a verified GitHub repository URL.	High	Pass: the profile is published to the catalog only after the GitHub URL passes validation. Fail: a malformed URL is accepted or the profile is published without validation.	Core feature that lets teams present their projects in the expo catalog.
FR-002	Functional	The system shall allow student teams to upload multimedia artifacts such as posters, images, and demo videos in approved formats.	High	Pass: files in allowed formats (JPG, PNG, MP4) up to 100 MB upload and display on the profile. Fail: an unsupported or oversized file is accepted.	Expos are multimedia showcases, so visuals and video are essential.
FR-003	Functional	The system shall allow panel judges to grade assigned projects using a digital rubric with predefined, weighted criteria.	High	Pass: every rubric criterion is scored, the weighted total is calculated and stored, and submission is blocked until all criteria are scored. Fail: submission succeeds with a missing criterion.	Ensures a structured, consistent, and fair evaluation.
FR-004	Functional	The system shall allow a member of the public to cast exactly one vote per verified identity for a showcased project.	High	Pass: a verified user's first vote is recorded and a second vote from the same identity is rejected. Fail: duplicate votes from one identity are counted.	Enables public participation while protecting the integrity of the vote.
FR-005	Functional	The system shall display a leaderboard that ranks projects by a combined score of judge grades and public votes.	Medium	Pass: after a new valid vote or judge score, the leaderboard displays the updated ranking in the correct order within 300 ms. Fail: the ranking is incorrect or is not updated within 300 ms.	Drives engagement and gives transparent standings.

Req ID	Type	Description	Priority	Acceptance Criteria	Rationale
NFR-001	Non-functional (Performance)	The public voting and leaderboard calculations shall update in real time, with an API response latency under 300 ms.	High	Pass: benchmarking under simulated peak load confirms a 95th-percentile API latency below 300 ms. Fail: the latency is 300 ms or higher under peak load.	Ensures a responsive experience during high-traffic expo periods.
NFR-002	Non-functional (Security)	The system shall enforce anti-sybil protection so that every public voter is uniquely verified (for example, through email OTP or SSO) before a vote is counted.	High	Pass: automated tests block at least 99% of duplicate or bot accounts before they can vote. Fail: sybil or bot accounts are able to cast counted votes.	Protects the credibility of the public voting results.

3. UML Use-Case Diagram

The diagram below shows the three actors, the primary and supporting use cases, and the relationships between them. It includes two «include» relationships and one «extend» relationship.

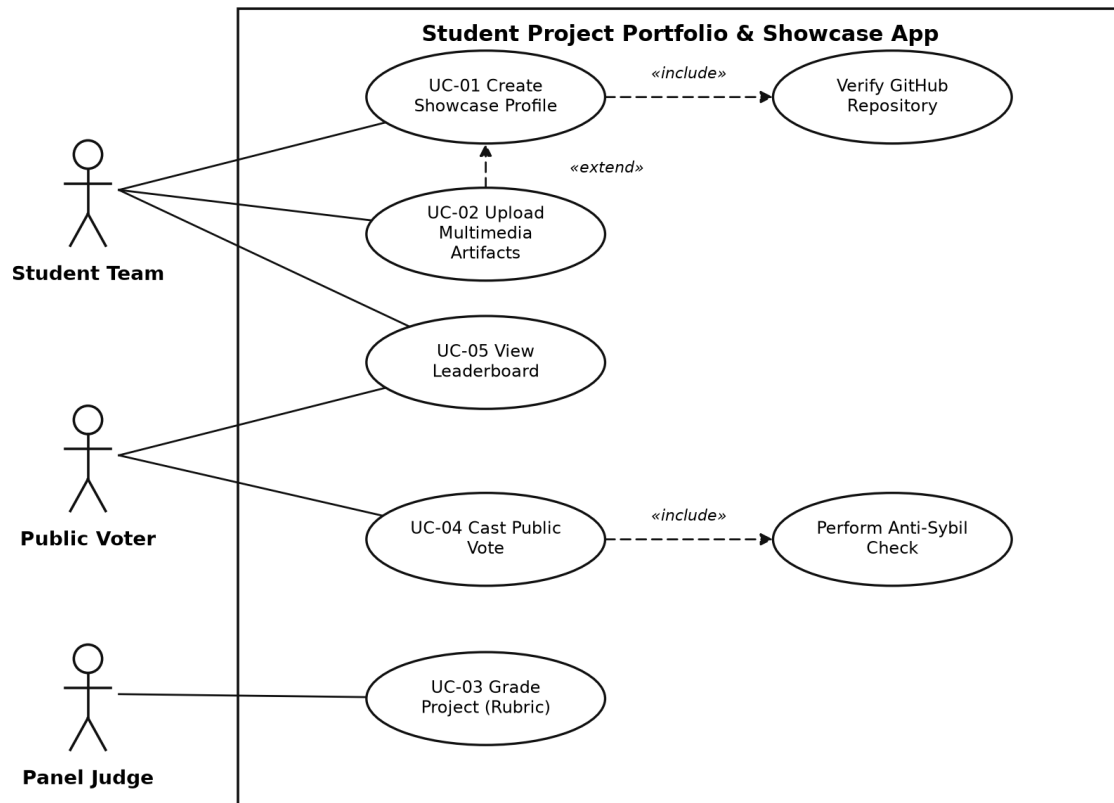


Figure 1. Use-case diagram for the Student Project Portfolio and Showcase App.

Relationships

- «include»: UC-01 Create Showcase Profile includes Verify GitHub Repository, because repository validation always runs while a profile is created.
- «include»: UC-04 Cast Public Vote includes Perform Anti-Sybil Check, because identity verification always runs before a vote is counted.
- «extend»: UC-02 Upload Multimedia Artifacts extends UC-01 Create Showcase Profile, because a team may optionally add posters, images, or demo videos while creating the profile.

4. Use-Case Flow Specification

Use Case: UC-01 Create Showcase Profile

Primary Actor: Student Team

Preconditions

- The student team is logged in to the system.
- The expo submission window is open.

Postconditions

- A showcase profile is created and published to the catalog.
- The GitHub repository URL is validated and linked to the profile.

Main Success Scenario

1. The student team selects the option to create a showcase profile.
2. The system displays the profile form (abstract summary, demo video link, and GitHub repository URL).
3. The student team enters the abstract summary and the demo video link.
4. The student team enters the GitHub repository URL.
5. The system validates the GitHub repository URL (include: Verify GitHub Repository).
6. The system confirms that the repository is valid, public, and accessible.
7. The student team submits the profile.
8. The system publishes the profile to the showcase catalog.
9. The system shows a confirmation message with the public profile link.
10. The use case ends successfully.

Extensions

- Upload Multimedia Artifacts (UC-02): before submitting, the student team may optionally upload posters, images, or demo videos to the profile. This optional step is modelled as an «extend» of this use case.

Alternate Flow - 5a. Invalid or inaccessible GitHub repository URL

- 5a1. The system displays the message: Invalid repository URL. Please provide a valid, public GitHub link.
- 5a2. The profile is not published.
- 5a3. The student team corrects the URL and resubmits (return to step 5).
- 5a4. If validation still fails after three attempts, the system saves the profile as a draft so the team can complete it later.